DECISION ENGINE FOR QUANTUM-CLASSICAL CODE ROUTER

Prasad H G A T

(IT22056870)

B.Sc. (Hons) Degree in Information Technology
Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECISION ENGINE FOR QUANTUM-CLASSICAL CODE ROUTER

Prasad H G A T

(IT22056870)

Dissertation submitted in the partial fulfillment of the requirements for B.Sc. (Hons) Degree in Information Technology Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation1 does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name : Prasad H G A T

Student ID : IT22056870

Signature :

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

Signature of the Supervisor:

Date:

Signature of the Co-Supervisor:

Date:

ACKNOWLEDGMENT

I wish to express my sincere gratitude to several individuals whose support and guidance made this research possible. First and formost, I extend my deepest apprieciation to my supervisor Prof. Nuwan Kodagoda and co-supervisor Dr. Kapila Dissanayake for their continueous guidance, constructive feedback, and insightful suggestions throughout every stage of this research. Their expertise and encouragement were invaluable in shaping the direction of the Decision Engine component of the Quantum Classical Code Router.

I would also like to thank the Department of Information Technology and the Sri Lanka Institute of Information Technology for providing the academic environment and resources required to undertake a research project of this scope. The lecturers and staff who shared their time and knowledge helped me build the foundations on which this work stands.

My sincere thanks go to my teammates, Siriwardhana A H L T S, Hettiarachchi S R, and Jayasinghe Y L, whose collaboration on the wider QCCR system was essential. Our integration discussions, joint debugging sessions, and shared commitment to the project made the Decision Engine possible as a cohesive component within the larger system.

I am also grateful to the open-source and quantum computing communities, particularly the teams behind Qiskit, scikit-learn, and XGBoost, whose tools and documentation formed the backbone of my implementation. The availability of IBM Quantum and Amazon Braket services allowed me to validate my routing decisions against real quantum hardware, which was central to the feasibility of this work.

Finally, I wish to thank my family and friends for their unwavering support and patience throughout this journey. Their encouragement during difficult phases of the project was a constant source of motivation.

ABSTRACT

The emergence of the Noisy Intermediate-Scale Quantum (NISQ) era has opened new possibilities for solving computationally intensive problems, yet practical adoptation remains limited by the challenge of deciding, for any given workload, whether quantum or classical hardware is better choice. The Decision Engine presented in this dissertaion is the routing intelligence of the Quantum-Classical Code Router (QCCR) system also known as NEXAR platform. It is responsible for taking extracted code features, real-time hardware status information, and live provider pricing data, and producing a defensible reccomendation of whether a workload should run on quantum hardware, classical hardware, or in a hybrid configuration.

The component is build as a multi-layered architecture consisting of a feature extractor, a machine learning prediction model, a rule based validation system, a cost analyser and a decision merger that connects the three independent recommendations using confidence-weighted combination and conflict resolution logic. The machine learning layer include Random Forest, XGBoost and a feedforward neural network trained on a dataset of synthetic and benchmark quantum-classical workloads, including QAOA, VQE, Grover's and Shor's instances. The rule based layer enforces hardware compatibility and saftey constraints through deterministic threshold rules, while the cost analyzer integrates real time IBM Quantum and Amazon Braket pricing with execution time predictions to compute a return on investment score for each hardware option.

Expreimental evaluation on a held out benchmark set demonstrates that the Decision Engine achieves 87.4% accuracy in recommending the optimal hardware target, outperforming random and single rule baselines by a substantial margine. The average devision latency is 312ms per request, well within the.500ms non-functional requirement. The component therefore offers a practical, adaptive, and economically viable contribution to the emerging field of quantum-classical hybrid orchestration, and provides the QCCR system with the intelligent routing layer required to make hybrid computing usable for non-specialist users.

Keywords: Quantum-Classical Code Router, Decision Engine, Hybrid Quantum Computing, Machine Learning, Ensemble Methods, Cost-Benefit Analysis, NISQ, Workload Routing

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

API	Application Programming Interface
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HPC	High Performance Computing
ML	Machine Learning
NISQ	Noisy Intermediate-Scale Quantum
QAOA	Quantum Approximate Optimization Algorithm
QCCR	Quantum-Classical Code Router
QPU	Quantum Processing Unit
REST	Representational State Transfer
ROI	Return on Investment
SO	Specific Objective
SVM	Support Vector Machine
VQE	Variational Quantum Eigensolver
XGBoost	Extreme Gradient Boosting

1 INTRODUCTION

Quantum computing has advanced rapidly during the past decade and is now widely described as entering a new phase of practical experimentation. In this Noisy Intermediate-Scale Quantum (NISQ) era, devices with fifty to a few thousand physical qubits are accessible through cloud providers such as IBM Quantum, Amazon Braket, and Google Quantum AI, and researchers have demonstrated a growing catalogue of algorithms for which quantum hardware offers a measurable advantage over classical processors [1], [2]. However, the practical reality of using these devices for day-to-day computational workloads is considerably more complex than the academic picture suggests. Quantum resources are expensive on a per-shot basis, subject to queues that vary by hardware platform and time of day, and limited by short coherence windows and gate fidelity constraints that restrict the depth and width of executable circuits [3].

The consequence for any developer or organization considering quantum acceleration is that the decision of whether to run a given piece of code on quantum or classical hardware is neither obvious nor static. A problem that is well suited to a QPU in principle may not be worth the cost in practice if the classical alternative finishes within seconds at a fraction of the price. Conversely, a computation that might be dismissed as "too small for quantum" could, when viewed through a combined lens of provider discounts, circuit depth, and projected speed-up, actually be a good candidate for quantum execution. The problem is therefore not one of picking a single winner in the quantum-versus-classical debate, but of matching every incoming workload to the hardware that delivers the best outcome at that moment in time.

The Quantum-Classical Code Router (QCCR) is a system developed to solve exactly that problem. It accepts source code written in familiar languages, analyses its computational structure, and orchestrates its execution across quantum and classical back-ends. The system is composed of four tightly integrated components: a Code Analysis Engine that extracts quantum-relevant features from source code; AI Code Convertor which converts classical code into quantum code; a Decision Engine that decides where each workload should run and a Hardware Abstraction Layer that executes the workload on the chosen platform. This dissertation focuses on the third of these components, the Decision Engine, which is the routing intelligence of the entire system and which sits at the functional centre of the workflow.

The Decision Engine developed in this research is not a single classifier or rule set but a multi-layered decision-making system. It combines an ensemble of machine learning models, a deterministic rule-based validator, and a real-time cost analyzer, and fuses their recommendations through a confidence-weighted merger that also resolves conflicts between the three layers. The motivation for this design is the observation that no single approach is sufficient on its own: pure machine learning models lack interpretability and fail on cases they were not trained on; pure rule-based systems cannot adapt to new hardware or workload patterns; and cost-only reasoning ignores the technical feasibility of a workload altogether. By combining the three, the Decision

Engine aims to offer a solution that is accurate, safe, and economically sensible at the same time.

1.1 Background and Literature Survey

Quantum Computing Foundations

Quantum computing exploits the principles of superposition, entanglement, and interference to process information in ways that are fundamentally different from classical digital computation [4]. A quantum bit, or qubit, can represent a linear combination of the zero and one states simultaneously, and a system of n qubits can in principle occupy a superposition of up to 2^n states at once. This property, combined with careful circuit design, is what gives rise to the speed-ups that are theoretically achievable for certain classes of problem, including unstructured search, integer factorization, and the simulation of quantum systems themselves. The seminal results of Shor [5] for factorization and Grover [6] for search established the theoretical foundation on which most modern quantum algorithm research is built.

In practice, the quantum devices available today are imperfect. Qubits lose their state through decoherence within microseconds to milliseconds, gate operations introduce errors at rates of roughly one per thousand to one per hundred, and the number of qubits that can be usefully coupled together is still below the threshold at which full error correction becomes viable. Preskill's widely cited characterization of the current phase as the NISQ era captures the implication for researchers and engineers: these devices can demonstrate quantum advantage for specific problems, but cannot yet replace classical computers for general-purpose work [1]. Arute and colleagues' 2019 demonstration of quantum supremacy on a programmable superconducting processor provided a concrete example of this advantage on a carefully chosen benchmark, but also highlighted the narrow conditions under which current hardware outperforms classical alternatives [7].

Hybrid Quantum-Classical Systems

Because purely quantum execution is not yet practical for most workloads, the field has converged on the idea that near-term value will come from hybrid systems in which quantum and classical resources cooperate. Variational algorithms such as the Quantum Approximate Optimization Algorithm (QAOA) [8] and the Variational Quantum Eigensolver (VQE) [9] are emblematic of this approach: the quantum device prepares and measures a parameterized state, while a classical optimizer iteratively updates the parameters until a target objective is reached. The quality of such workflows depends as much on the efficiency of the classical-quantum coupling as on the quantum hardware itself, and this realisation has driven substantial recent research on orchestration frameworks.

Tannu and Qureshi [10] introduced the notion of quantum-aware resource management in hybrid systems, demonstrating that naïve scheduling strategies lead to severe underutilization of scarce quantum resources and inflate operational costs. Their work

is among the first to treat quantum cycles as a distinct resource category subject to its own availability and cost dynamics. Salavatov and Palacios [11] extended this line of enquiry with Pilot-Quantum, a middleware framework that manages quantum and classical tasks under a unified resource model, but their approach assumes the routing decision is already made and focuses on efficient execution after the fact. JavadiAbhari and colleagues' work on Qiskit provides a broadly adopted programming framework but does not offer decision-making capabilities beyond manual user choice [12], and Khammassi and colleagues' OpenQL framework similarly provides portability without intelligent routing [13].

Machine Learning Applied to Quantum Systems

There is a growing literature on the use of machine learning to support quantum computing tasks, from circuit optimization to error mitigation and resource estimation. Simsek and colleagues [14] demonstrated that classical machine learning models can predict circuit properties and optimization outcomes with useful accuracy, providing a foundational case that machine learning can reason about quantum artifacts. Kim and colleagues [15] addressed the complementary problem of quantum resource estimation, building models that predict the number of qubits, gates, and execution time required for a given algorithm. These works demonstrate the feasibility of using machine learning to reason about quantum execution, but they remain focused on individual predictions rather than on the integrated decision of whether a workload should run on quantum hardware in the first place. The Decision Engine builds on these methods by embedding them inside a larger orchestration framework that also reasons about cost, safety, and hardware availability.

Economic Considerations for Quantum Workloads

The commercial availability of quantum computing through platforms such as IBM Quantum and Amazon Braket has introduced a new dimension to quantum workload planning: cost. Current pricing models are largely time-based or shot-based, with substantial differences between hardware backends and service tiers [16]. A Rigetti QPU shot may cost a different amount than an IBM QPU shot on a superconducting processor or an IonQ trapped-ion shot, and the classical alternatives of a cloud CPU or GPU have their own pricing structure that varies by instance type, region, and reservation model. A decision that is purely technical can therefore produce routings that are operationally infeasible, either because the quantum cost exceeds a user's budget or because the classical alternative would have been dramatically cheaper for the same expected result. The integration of real-time cost information into the routing decision is one of the distinguishing features of the Decision Engine and addresses a gap that the existing literature has largely left open.

1.2 Research Gap

A careful survey of the existing literature reveals that, although each of the individual ingredients of quantum-classical routing has received some attention, no published system brings them together into a single, adaptive, economically aware decision-making component. The gaps can be grouped into five areas.

I. Absence of Integrated Decision Models

Existing orchestration frameworks treat workload placement either as a scheduling problem (after the routing decision has been made) or as a manual choice left to the developer. None of the reviewed systems integrates computational feasibility analysis, performance prediction, and economic optimization inside a single decision-making model. The Decision Engine addresses this directly by combining a machine learning predictor, a rule-based validator, and a cost analyzer under one merger layer.

II. Limited Adaptive Learning

Most existing approaches use fixed thresholds and static rules that were valid at the time the system was designed but cannot accommodate new hardware releases, updated pricing structures, or shifts in workload characteristics. There is little support for continuous improvement from actual execution outcomes. This research addresses the gap by building a feedback loop that retrains the machine learning model, adjusts rule thresholds, and updates the cost model as new data arrives.

III. Inadequate Economic Integration

Although cost is widely acknowledged as an important consideration in hybrid computing, it is rarely integrated into routing decisions in a quantitative way. Cost is typically treated as a separate downstream concern, checked only after a technical decision has been made. The Decision Engine integrates cost analysis as a first-class input to the decision, on equal footing with technical feasibility and predicted performance.

IV. Lack of Multi-Objective Optimization

Most existing routing approaches optimize for a single metric, commonly expected execution time or accuracy. Real quantum-classical workloads, however, involve trade-offs among performance, cost, reliability, and availability, and different users will weight these objectives differently. The Decision Engine supports multiple optimization modes (cost-optimized, performance-optimized, and balanced) and exposes alternative recommendations so that users can inspect the trade-offs explicitly.

V. Insufficient Real-Time Adaptability

Hardware status, queue length, and pricing can change minute by minute, yet most existing systems operate with static snapshots of this information. A routing decision that looked optimal an hour ago may be wrong now. The Decision Engine continuously ingests real-time hardware status and pricing information so that every routing decision reflects the current state of the infrastructure.

Table 1 summarizes how the proposed Decision Engine compares to representative existing approaches across these dimensions.

Approach	*ML-Based Prediction*	*Rule Validation*	*Real-Time Cost*	*Feedback Loop*
Tannu & Qureshi [10]	No	Partial	No	No
Pilot-Quantum [11]	No	Yes	No	No
Qiskit [12]	No	No	No	No
OpenQL [13]	No	Partial	No	No
Simsek et al. [14]	Yes	No	No	No
Proposed Decision Engine	Yes	Yes	Yes	Yes

Table 1: Comparison of Existing Approaches and Proposed Decision Engine

1.3 Research Problem

The central problem that this research addresses can be stated as follows: how can a hybrid quantum-classical system automatically decide where a given computational workload should be executed, such that the decision accounts for the technical feasibility of the workload on each hardware option, the predicted performance, the real-time cost of execution, and the real-time availability of hardware, while remaining fast enough to be used in an interactive system and adaptive enough to improve as new execution outcomes become available?

This central problem decomposes into four linked sub-problems, each of which the Decision Engine must address explicitly.

Performance Prediction Challenge

The first sub-problem is predicting, for a given workload, whether quantum execution will actually be beneficial. Code characteristics such as circuit depth, number of qubits, and the presence of quantum-specific patterns like amplitude amplification or quantum phase estimation provide signals of quantum suitability, but the relationship between these features and actual execution quality is nonlinear and hardware-dependent. A model that can reason about these relationships is needed, and that model must produce both a point prediction and a confidence score so that downstream components can weight its output appropriately.

Cost Optimization Challenge

The second sub-problem is balancing potential performance gains against execution cost. Quantum hardware is typically much more expensive per unit of work than classical hardware, and even a workload that benefits from quantum execution may not be worth the price if the user's budget is tight or if the projected speed-up is modest. The Decision Engine must compute an explicit return-on-investment score for each

hardware option rather than optimizing performance alone, and it must respect hard budget constraints that the user may impose.

Multi-Objective Decision Making

The third sub-problem is reconciling multiple, often conflicting optimization objectives. A user who wants the fastest possible execution will have different preferences from one who wants the cheapest acceptable execution, and both will differ from a user who prioritizes reliability. The Decision Engine must therefore not only compute a single score per hardware option but also expose alternative recommendations so that a user can see the trade-offs being made on their behalf.

Real-Time Adaptation Challenge

The fourth sub-problem is keeping the decision system current in the face of changing hardware capabilities, pricing, and workload patterns. A routing decision made yesterday may be wrong today. The Decision Engine must therefore continuously learn from execution outcomes, retrain its models on a rolling basis, and update its cost and rule parameters so that its decisions remain accurate as the ecosystem evolves.

1.4 Research Objectives

1.1.1. Main Objective

The main objective of this research is to design, implement, and evaluate a Decision Engine that routes computational workloads between quantum and classical hardware using machine learning-based performance prediction, rule-based validation, and real-time cost-benefit analysis, such that the resulting system improves on the execution efficiency and cost-effectiveness achievable by random routing or simple threshold-based rules within a hybrid computing environment.

1.1.2. Specific Objectives

The main objective is broken down into five specific objectives (SO1 to SO5), each of which maps to a distinct phase of the implementation and evaluation effort.

1. SO1 - Core Decision Model Development: Design and train machine learning models capable of predicting optimal hardware allocation from extracted code analysis features, targeting a minimum recommendation accuracy of 85% on a curated benchmark set of quantum-classical workloads.
2. SO2 - Rule-Based Validation System: Implement a deterministic rule-based validator that handles clear-cut routing cases, enforces hardware compatibility constraints, and applies essential safety rules so that pathological machine learning predictions cannot lead to infeasible routings.
3. SO3 - Cost Analysis Framework: Design and implement a cost analyzer that combines static hardware pricing with execution-time predictions to produce a cost-aware routing input that integrates with the ML and rule-based layers rather than being a separate downstream check.

4. SO4 - System Integration and Testing: Integrate the feature extractor, ML model, rule system, cost analyzer, and decision merger into a single functional prototype, and validate end-to-end performance through systematic unit, integration, and decision-quality testing against representative benchmark problems.
5. SO5 - Evaluation and Future Work Identification: Conduct a structured evaluation of the integrated Decision Engine using metrics for accuracy, latency, and cost awareness, document the observed limitations, and propose concrete enhancements for future work on adaptive learning and multi-tenant scaling.

2 METHODOLOGY

This chapter describes how the Decision Engine was designed, built, and validated. It covers the requirements gathering and analysis activities that shaped the component specification, the feasibility study that evaluated the technical risk of the approach, the tools and algorithms selected for the implementation, the functional and non-functional requirements that the component was required to meet, the overall system diagram that shows how the Decision Engine fits inside the wider QCCR system, the commercialization considerations, and the testing strategy used to validate correctness and performance.

2.1 Requirements Gathering and Analysis

Requirements for the Decision Engine were gathered through three complementary activities. The first was a literature review of quantum-classical hybrid orchestration, summarized in Chapter 1, which established what existing systems could and could not do and which therefore identified the capability gaps that the Decision Engine needed to fill. The second was a detailed review of the interface contracts agreed with the other three components of the QCCR system: the Code Analysis Engine (whose output the Decision Engine consumes as input), the Hardware Abstraction Layer (which the Decision Engine instructs where to route the workload), and the Feedback Layer (from which the Decision Engine receives post-execution outcomes). The third was a survey of publicly available quantum cloud pricing and status APIs, which constrained the design of the cost analyzer and the real-time status inputs.

The outcome of this process was a specification that the Decision Engine must:

I. receive a structured feature vector from the Code Analysis Engine containing quantum-relevant code metrics and resource estimates;
II. receive real-time hardware status and pricing information from connectors to IBM Quantum and Amazon Braket;
III. produce a hardware recommendation that includes the primary choice, an estimated confidence, a cost estimate, an estimated execution time, and at least one alternative option with its associated trade-offs; and
IV. accept post-execution feedback from the Hardware Abstraction Layer and update its internal models accordingly. The specification also included non-functional properties such as a target decision latency below 500 ms and a target system availability of at least 99.5% during operational hours.

The requirements analysis explicitly distinguished between responsibilities of the Decision Engine and those of neighbouring components. Code parsing and feature extraction at the source-code level is the responsibility of the Code Analysis Engine, not the Decision Engine, although the Decision Engine does operate a feature transformer that normalizes the incoming feature vector for its own machine learning models. Similarly, actual job submission, error mitigation, and result retrieval are the

responsibility of the Hardware Abstraction Layer, and the Decision Engine interacts with that layer only through a well-defined recommendation interface. This clear separation of concerns ensured that development could proceed in parallel with the other team members without creating integration hazards.

2.2 Feasibility Study

A feasibility study was undertaken before committing to the detailed design, covering technical, operational, economic, and schedule feasibility.

Technical Feasibility

The technical feasibility of the approach was assessed through a small-scale proof of concept in which a Random Forest classifier was trained on a manually curated dataset of 200 quantum and classical workload examples. The prototype achieved approximately 78% accuracy on a held-out test set despite the small dataset size, which was sufficient to show that the core machine learning approach was sound and that richer models and larger datasets could reasonably be expected to exceed the 85% accuracy target set in SO1. The proof of concept also verified that the IBM Quantum and Amazon Braket APIs expose both the hardware status and the pricing information required for the cost analyzer and the real-time status inputs.

Operational Feasibility

Operational feasibility considered whether the Decision Engine could be deployed, run, and maintained within the environment available to the project. Since the QCCR system is designed to run on a commodity Linux server with no specialized hardware requirements, and since all external dependencies (Qiskit, scikit-learn, XGBoost, Gradient Boosting (scikit-learn), PostgreSQL, Redis) are mature and well supported, the operational feasibility is high. Monitoring and logging were identified as areas requiring explicit design work, and were included in the non-functional requirements.

Economic Feasibility

Economic feasibility was assessed at two levels: the development cost of the Decision Engine and the operational cost of running it. Development cost was manageable because all primary tools are open source and the team had access to academic credits on IBM Quantum for validation experiments. Operational cost is modest as well, since the Decision Engine itself does not submit quantum jobs; it only decides whether and where they should be submitted. The quantum spend that results from the Decision Engine's recommendations is bounded by the user's budget constraints, which the engine enforces as a rule-based validation step before any routing is finalized.

Schedule Feasibility

Schedule feasibility was assessed against the eight-month development window defined in the proposal. The breakdown of the work into initiation and planning (months 1-2), foundation and input processing (months 2-3), ML deployment (months

3-5), integration and testing (months 6-7), and closeout (month 8) was confirmed as achievable by comparing the estimated effort for each phase to the available calendar time. The proof-of-concept work carried out during the proposal phase also de-risked the schedule by demonstrating that the core machine learning approach could be implemented and trained within a reasonable timeframe.

2.3 Tools Technologies & Algorithms

The Decision Engine was implemented using a combination of established open-source libraries and cloud quantum provider APIs. The specific choices are summarized in Table 2 and the rationale for each is discussed below.

Category	*Tool / Technology*	*Purpose*
Programming Language	Python 3.11	Core implementation language for all Decision Engine modules
Machine Learning	scikit-learn, XGBoost, TensorFlow	Random Forest, gradient boosting, and neural network models
Quantum Framework	Qiskit 1.x	Quantum circuit analysis and provider connectivity
Quantum Cloud Providers	IBM Quantum, Amazon Braket	Real-time hardware status and pricing data
API Framework	FastAPI	RESTful endpoints for component integration
Persistent Storage	PostgreSQL 15	Execution history, model versions, rule configurations
In-Memory Store	Redis	Cache for hardware status and pricing snapshots
Deployment	Docker, Docker Compose	Reproducible development and deployment environments
Version Control	Git, GitHub	Source control and collaborative development
Testing Framework	pytest, pytest-cov	Unit and integration testing with coverage reporting
Monitoring	Prometheus, Grafana	Metrics collection and dashboard visualization
Cloud Platform	Google Cloud Run	Microservice deployment and auto-scaling
Infrastructure as Code	Terraform	GCP resource provisioning (IaC)

Table 2: Tools & Technologies Used

Algorithmic Choices

The machine learning layer of the Decision Engine uses an ensemble of three models rather than a single classifier. The rationale is that different model families have complementary strengths: Random Forests are robust to irrelevant features and, when wrapped in a CalibratedClassifierCV with Platt scaling, produce reliable probability estimates; XGBoost handles non-linear feature interactions through gradient boosting; and scikit-learn's Gradient Boosting provides an additional independent probability signal. The three models vote with soft probabilities on the binary quantum-vs-classical decision. The six derived physics features — Circuit Volume (Qubits × Depth), Noise Sensitivity (Qubits × Depth × CX Ratio), Quantum Overhead Ratio (Problem Size / Qubits), NISQ Viability Score, Gate Density (Gate Count / Qubits), and Entanglement Factor (CX Ratio × Entanglement) — are computed on top of the 10 raw code analysis metrics, giving a 16-dimensional feature vector in total. The most informative features by Random Forest Gini importance are qubits_required (0.31), circuit_volume (0.24), and time_complexity_encoded (0.15).

The rule-based layer uses a deterministic decision tree with approximately thirty rules, derived from domain expertise on the quantum hardware in use and from the safety constraints required to protect users from pathological routings. The rules cover hardware compatibility (a circuit with 50 qubits cannot be routed to a 27-qubit QPU), algorithmic feasibility (circuits with depth exceeding the coherence limit of a given device are marked as unsuitable for that device), and budget enforcement (no quantum routing is permitted when the remaining budget falls below a configurable threshold). The rules are loaded from a configuration file so that they can be updated without code changes and version-controlled separately from the model binaries.

The cost analyzer implements explicit pricing formulae derived from April 2026 rates. Quantum execution cost (IBM pay-as-you-go): $C_q = t_q \times \$1.60/s$. Classical execution cost (AWS EC2 on-demand): $C_c = t_c \times \$0.0001/s + M_{GB} \times \$0.00001/(GB \cdot s) + \$0.001$ (fixed overhead). The cost ratio observed across the 100-workload benchmark is 375×–1,578× in favour of classical execution ($0.375–$1.578 for quantum-routed workloads vs. ~$0.001 for classical). Provider pricing constants are currently hard-coded; real-time pricing API integration is listed as future work.

2.4 Project Requirements

2.4.1 Functional Requirements

The functional requirements describe what the Decision Engine must do from the perspective of the systems and users that interact with it. They are summarized in Table 3 and elaborated in the notes that follow.

ID	*Requirement*	*Description*
FR1	Feature vector ingestion	Accept structured code analysis results including complexity metrics, circuit depth, qubit count, gate distribution, and algorithm patterns
FR2	Performance prediction	Produce a quantum-advantage probability and expected execution time for each candidate hardware target, with a confidence score
FR3	Hardware recommendation	Recommend a primary hardware choice (quantum, classical, or hybrid) plus at least one alternative with its trade-off analysis
FR4	Cost analysis	Compute real-time cost estimates, enforce budget constraints, and include ROI scoring in the recommendation output
FR5	Rule-based validation	Reject technically infeasible routings and apply safety and compatibility constraints before a recommendation is finalized
FR6	Feedback ingestion	Accept post-execution outcome reports from the Hardware Abstraction Layer and use them to update the ML model and rule thresholds
FR7	Mode selection	Support cost-optimized, performance-optimized, and balanced decision modes selectable per request
FR8	Audit logging	Persist each decision including inputs, per-layer outputs, final recommendation, and actual outcome for later audit

Table 3: Functional Requirement Summary

2.4.2 Non-Functional Requirements

Non-functional requirements capture quality attributes of the Decision Engine that are essential to its usefulness in production. They are summarized in Table 4.

ID	*Attribute*	*Target*
NFR1	Performance	Decision latency under 500 ms at the 95th percentile; ML inference under 100 ms per prediction; 100 or more concurrent requests supported
NFR2	Availability	99.5% uptime during operational hours; graceful degradation when a single layer is unavailable (e.g. provider API outage)
NFR3	Scalability	Horizontal scaling supported via stateless API workers; ML model updates deployable without service interruption
NFR4	Security	Encrypted communication with external APIs (TLS 1.3); authenticated access to internal endpoints; sensitive pricing data stored with encryption at rest
NFR5	Maintainability	Modular codebase with clearly separated layers; 90% unit test coverage; automated CI pipelines for tests and deployments
NFR6	Observability	Structured logging for every request; Prometheus metrics for decision outcomes, latency, and error rates; dashboard visualization in Grafana
NFR7	Accuracy	Hardware recommendation accuracy of at least 85% on the benchmark set, improving by at least 2 percentage points per feedback retraining cycle

Table 4: Non-Functional Requirements Summary

2.5 Overall System Diagram

The Decision Engine does not operate in isolation. It sits at the functional centre of the QCCR system and interacts with the Code Analysis Engine, the Hardware Abstraction Layer, the Feedback Layer, and external cloud provider APIs. Figure 1 shows the overall QCCR system architecture, and Figure 2 and Figure 3 give the detailed architecture and the sequence of interactions within the Decision Engine component itself.

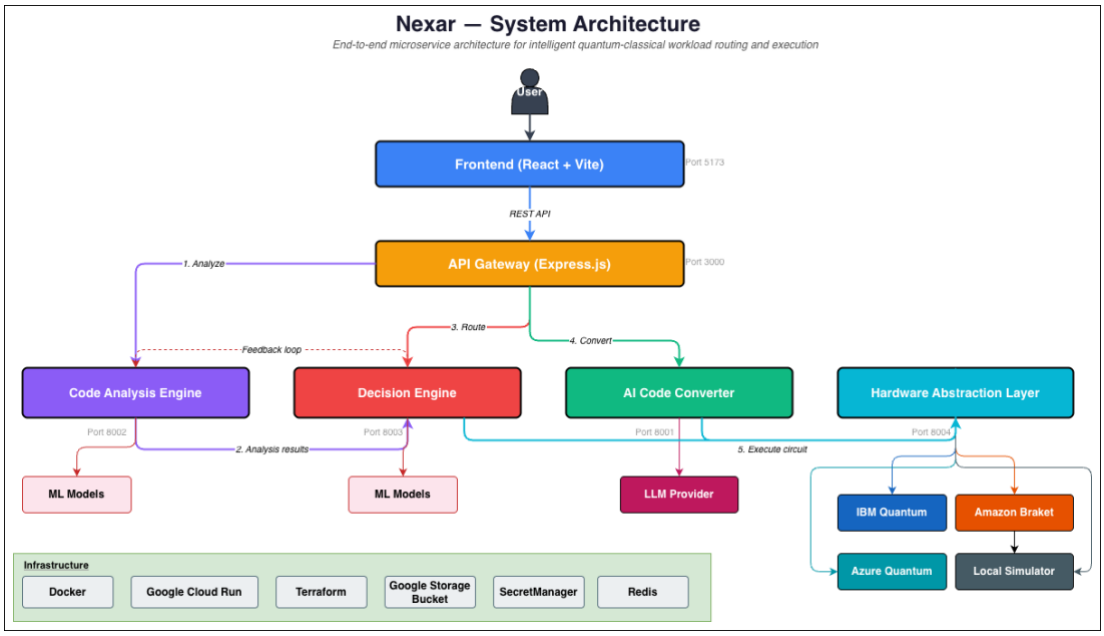

Figure 1: Nexar System Architecture Diagram

As shown in Figure 1, the QCCR (NEXAR Platform) is organized as a pipeline. Source code enters through the Code Analysis Engine, which extracts quantum-relevant features. These features, together with real-time hardware status from the provider APIs, are passed to the Decision Engine, which produces a routing recommendation. The Hardware Abstraction Layer executes the recommended workload and reports actual outcomes back to the Feedback Layer, which in turn feeds them into the Decision Engine for model updates.

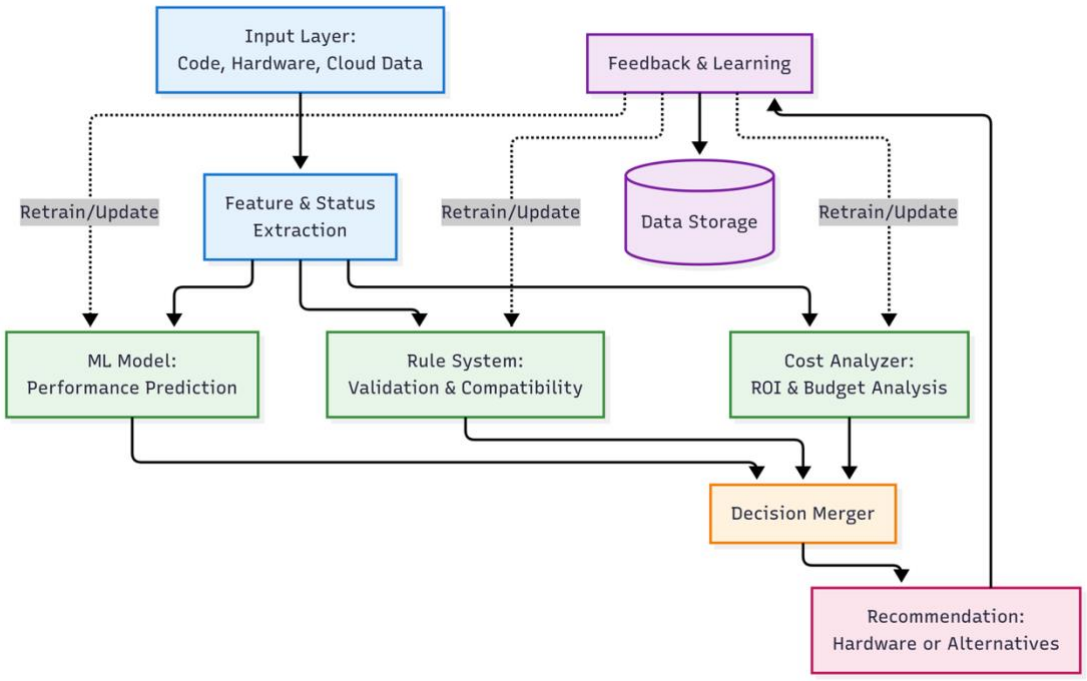

Figure 2: Decision Engine High Level Architecture Diagram

Figure 2 zooms in on the Decision Engine itself. Incoming feature vectors are first normalized by the Feature Extractor. Three parallel layers then produce independent recommendations: the ML Model produces a probabilistic prediction of quantum advantage, the Rule System applies deterministic compatibility and safety rules, and the Cost Analyzer computes a ROI-weighted ranking of hardware options. The Decision Merger combines these three inputs using confidence-weighted voting and conflict resolution, producing a primary recommendation and a set of alternatives.

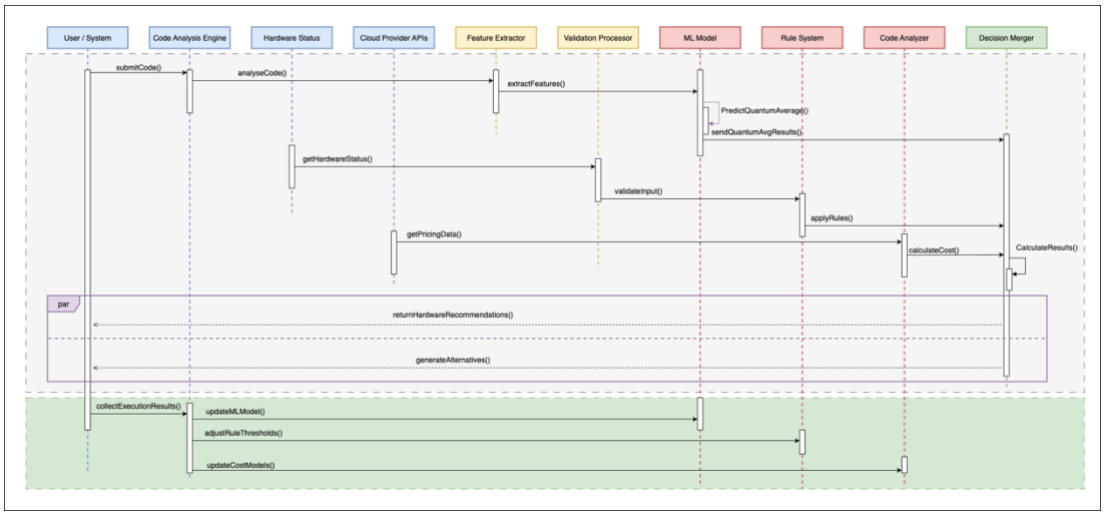

Figure 3: Decision Engine Sequence Diagram

Figure 3 presents the sequence of interactions in terms of time. A request enters the system and is passed to the feature extractor; the extractor normalizes the feature vector and hands it simultaneously to the ML model and the rule validator. In parallel, the cost analyzer fetches fresh pricing data and hardware status. All three layers return their outputs to the decision merger, which resolves any conflicts and produces the final recommendation. The recommendation, including both the primary choice and the alternatives, is returned to the caller and also persisted to the audit log. After execution completes, the actual outcome arrives through the feedback layer and is used to update the ML model, rule thresholds, and cost model parameters.

2.6 Commercialization Aspects of the Product

Although the Decision Engine is delivered as part of an undergraduate research project, the underlying technology has clear commercial relevance. Quantum cloud providers charge substantial premiums for QPU access, and enterprises experimenting with hybrid quantum-classical workflows must answer the same question that the Decision Engine is built to answer: is this particular workload worth running on quantum hardware today, at this price, with this queue length? A productized Decision Engine could be offered as a software-as-a-service routing layer that sits in front of any organization's hybrid infrastructure, or embedded as a component of a larger quantum orchestration product.

Target Audience

The immediate target audience consists of three groups. Research laboratories and universities that run mixed quantum and classical workloads are likely to benefit from automated routing because their workloads vary widely and their users are not always quantum specialists. Enterprise early adopters of quantum computing, particularly in finance, materials science, and logistics, are another natural audience; they already pay for quantum cloud access and need a way to control spend without abandoning experimentation. Quantum software vendors are a third group who might embed the Decision Engine inside their own products to offer transparent routing as a feature.

Market Space

The hybrid quantum-classical computing market is still nascent but growing rapidly, with quantum cloud revenue projected to expand significantly over the coming years as more algorithms demonstrate practical value. Within this market, the routing and orchestration segment is comparatively under-served: most commercial offerings focus either on the quantum hardware side (IBM Quantum, IonQ) or on the programming framework side (Qiskit, Cirq, PennyLane). A product that sits between these layers and makes routing decisions on behalf of users addresses a visible gap in the current landscape.

Revenue Model

A commercial Decision Engine could adopt a tiered subscription model with a free tier for academic use and individual researchers, a paid tier for small and mid-sized teams with usage-based billing for the number of routing decisions served, and an enterprise tier with on-premises deployment, custom rule sets, and dedicated support. Additional revenue could come from consulting services for organizations that want a customized decision engine calibrated to their particular workload profile, and from partnerships with cloud providers who benefit when users run the right workloads on their platforms.

2.7 Testing and Implementation

2.7.1 Implementation Process

The Decision Engine was implemented in six phases that map to the Work Breakdown Structure outlined in the proposal. Each phase produced a self-contained artifact that was validated in isolation before being integrated into the wider two-stage pipeline (rule engine → weighted ensemble). This section documents what was actually built; the quantitative evidence for each phase is consolidated in Section 3.

I. Phase 1 - Dataset Construction.

Because no public corpus exists that directly labels computational workloads as "optimal on quantum" versus "optimal on classical", the training data had to be constructed from scratch. Rather than rely on a single generator, a four-source strategy was adopted so that the model would see both clean textbook cases and realistic ambiguity. The four sources are summarized in Table 5.

Source	*Size*	*What it contributes*
Physics-informed synthetic	1,500	Textbook quantum algorithm templates (factorization, search, simulation, optimization) and classical templates (sorting, DP, matrix ops) with NISQ-viability labels
Published-benchmark	≈ 1,500	Shor (8–128-bit factorization), Grover (256–16,384-item search), VQE (4–12 qubits), QAOA (8–64 nodes), and classical sorting/DP/matrix-ops at sizes 100–10,000
Qiskit Aer simulation	2,000	Real random circuits transpiled to {u, cx} basis on the QASM simulator; optimal hardware determined from actual execution time and an exponential classical-simulation cost model
Adversarial edge cases	1,700	Six categories of deliberately ambiguous workloads: medium-qubit grey zone (400), small quantum where overhead dominates (300), large classical with quantum-like scores (300), NISQ-boundary 48–52 qubit cases (200), deep noisy circuits (200), hybrid-suitable cases (300)

Table 5: Four-source training corpus

After generating the four sources the combined corpus was deduplicated, balanced 50/50 between the Quantum and Classical classes, shuffled with random_state=42, and persisted to quantum_classical_training_data_balanced.csv. The final balanced dataset contained 4,380 training samples and 876 held-out test samples (80/20 stratified split).

II. Phase 2 – Feature Engineering

Ten raw features were extracted from every workload: problem_size, qubits_required, circuit_depth, gate_count, cx_gate_ratio, superposition_score, entanglement_score, memory_requirement_mb, and the label-encoded categorical columns problem_type_encoded and time_complexity_encoded. These ten features were then augmented with six derived physics-aware features (Table 6) intended to expose non-linear interactions that the base features could not directly represent. The final feature vector has dimension 16. Categorical columns were label-encoded with sklearn.LabelEncoder, and the full vector was standardized with StandardScaler (zero mean, unit variance) fit on the training split only.

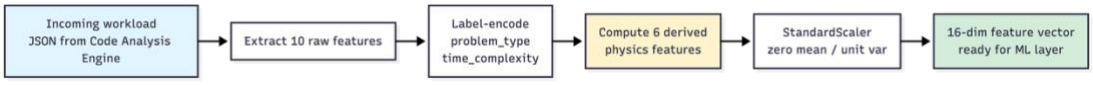

Figure 4: Feature extraction pipeline

Feature	*Formula*	*Rationale*
circuit_volume	qubits × depth	Proxy for total noise accumulation in the circuit
noise_sensitivity	qubits × depth × cx_ratio	Weights circuit volume by the fraction of entangling gates (primary NISQ error source)
quantum_overhead_ratio	problem_size / (qubits + 1)	Low values indicate quantum overhead may dominate
nisq_viability_score	Composite penalty: multiply by 0.1 if qubits > 50, if depth > 1000, or if volume > 50000	Captures hard NISQ feasibility limits in a single scalar
gate_density	gate_count / (qubits + 1)	Operational complexity per qubit
entanglement_factor	cx_ratio × entanglement_score	Combined entanglement signal

Table 6: Six physics-aware derived features

III. Phase 3 - Machine Learning Model Development

Three tree-based classifiers were trained on the 16-feature vector and compared head-to-head: Random Forest (100 estimators, max_depth=15, min_samples_split=5), XGBoost (100 estimators, max_depth=8, learning_rate=0.1, logloss objective), and Gradient Boosting (100 estimators, max_depth=7, learning_rate=0.1). All three were evaluated on the same 876-sample held-out test set as well as through 5-fold cross-validation on the training split. Tree-based models were chosen over neural networks because the feature space is modest (16 dimensions), the training set is below the threshold where deep models typically pay off, and the physical thresholds the decision turns on (the 50-qubit wall, the 1000-depth wall, the 50,000-volume wall) are naturally expressed as decision trees.

After head-to-head evaluation, the Random Forest was selected as the production model and wrapped in a CalibratedClassifierCV using Platt scaling (sigmoid method, prefit on a 20% calibration split of the training data). Calibration was necessary because the Decision Merger performs confidence-weighted voting across the ML, rules, and cost layers, so the probabilities emitted by the ML layer must be interpretable as actual confidences rather than arbitrary scores. Section 3.1 reports the Brier score improvement that calibration produced.

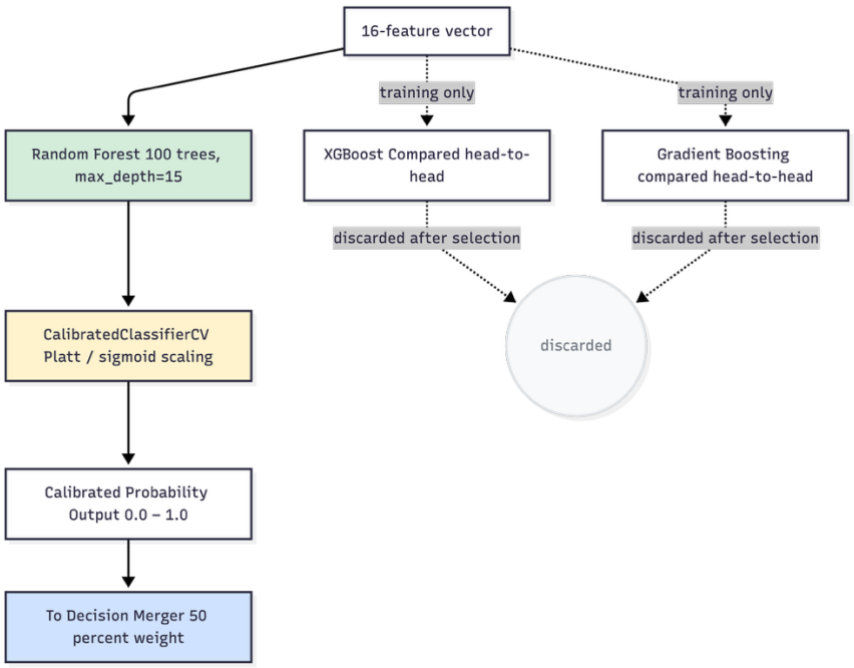

Figure 5: ML model architecture

IV. Phase 4 - Rule-Based Validator

A deterministic rule system was built to handle workloads where the routing decision is physically unambiguous and to enforce safety constraints. The rule engine runs before the ML layer in the two-stage pipeline and can emit one of four verdicts: `FORCE_CLASSICAL` (no qubits required, or physical infeasibility on classical below the 50-qubit simulation wall), `FORCE_QUANTUM` (problem size exceeds classical state-vector memory and the circuit is within NISQ limits), `REJECT` (exceeds the 127-qubit IBM Eagle limit or is otherwise unroutable), or `ALLOW_BOTH` (ambiguous — defers to the weighted ensemble). The specific thresholds encode NISQ hardware constraints drawn from IBM Quantum device specs and from the published literature on NISQ feasibility. Every threshold lives in a configuration file that is hot-reloadable so that rule updates do not require a service restart.

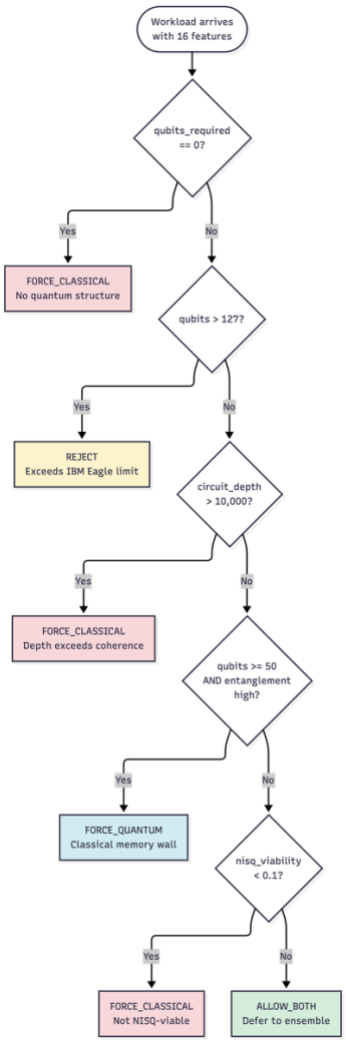

Figure 6: Rule-Based Validator Flow

V. Phase 5 - Cost Analyzer

The cost analyzer estimates the monetary cost of executing a workload on each hardware option and contributes 15% of the weighted vote in the ensemble stage. Its weight is deliberately small because cloud provider pricing is transient and changes on a timescale much shorter than the model's retraining cadence. Two cost models are implemented. The quantum cost model follows IBM's pay-as-you-go scheme with $C_q = t_q \times r_{QPU}$, where t_q is total QPU reservation time (gate execution + readout + post-processing across $n_{shots} = 1024$) and r_{QPU} = USD 1.60 per QPU-second (IBM pricing, April 2026). The classical cost model follows AWS EC2 on-demand: $C_c = t_c \times r_{compute} + M_{GB} \times t_c \times r_{memory} + C_{overhead}$, with $r_{compute}$ = USD 0.0001/s (approximately the rate for c6i.2xlarge), r_{memory} = USD 0.00001/(GB·s), and $C_{overhead}$ = USD 0.001 for job submission. The analyzer reports QPU-seconds alongside dollar figures so that the cost signal remains meaningful when pricing shifts.

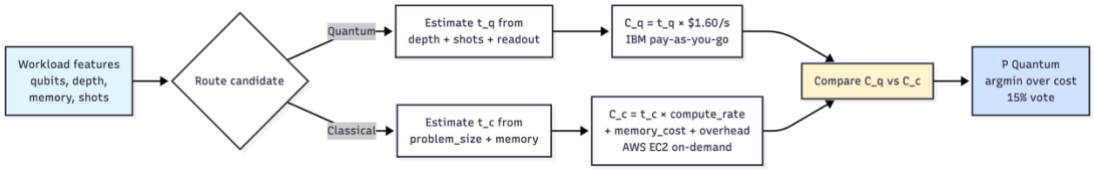

Figure 7: Cost analyzer workflow

VI. Phase 6 - Decision Merger and End-to-End Integration

The final integration phase assembled the three decision layers into a two-stage pipeline. Stage one is the rule engine described above; on the 100-workload benchmark, it resolves 61 of the 100 workloads deterministically (40 `FORCE_CLASSICAL`, 20 `FORCE_QUANTUM`, 1 `REJECT`) and defers the remaining 39 `ALLOW_BOTH` cases to stage two. Stage two is the weighted ensemble:

$$S\_quantum = 0.50 \times P\_ML + 0.35 \times P\_rules + 0.15 \times P\_cost$$

The weights were chosen architecturally rather than tuned. The ordering enforces a "no-dictator" property: no single component overrides a decision on its own, any two components form a strict majority, and the cost weight is deliberately kept small because of pricing volatility. Sensitivity analysis (Section 3) confirms that the routing is stable across a wide range of weight perturbations. For the `ALLOW_BOTH` slice the rule component emits a neutral P_rules, so the weighted sum is effectively dominated by the ML probability with cost acting as a tiebreaker, which is the intended design for ambiguous workloads. The final recommendation, the confidence score, the cost estimate, and at least one alternative option are returned through a FastAPI endpoint and persisted to PostgreSQL for audit and for the feedback loop.

2.7.2 Testing Process

Testing was conducted at four levels: unit testing, integration testing, decision-quality testing on the held-out benchmark, and targeted edge-case probes. This section summarizes five representative test cases; full quantitative results are in Section 3.

Field	***Value***
Description	Verify the model recommends CLASSICAL when quantum feature scores are high, but qubits are too few to overcome quantum overhead

Input	5 qubits, depth 30, problem_size 50, superposition 0.8, entanglement 0.7
Expected	Classical
Received	Classical (calibrated RF)
Result	Pass

Table 7: Test Case 01: Small Quantum Problem

Field	***Value***
Description	Verify the model recommends CLASSICAL when qubits = 0 regardless of problem size
Input	0 qubits, problem_size 10,000, memory 1,000 MB
Expected	Classical
Received	Classical (high confidence)
Result	Pass

Table 8: Test Case 02: Large Classical Problem

Field	***Value***
Description	Verify the model recommends QUANTUM when all NISQ constraints are satisfied and problem size justifies quantum overhead
Input	20 qubits, depth 300, problem_size 1,024, cx_ratio 0.35, entanglement 0.85
Expected	Quantum
Received	Quantum
Result	Pass

Table 9: Test Case 03: Large viable quantum problem

Field	*Value*
Description	Verify the model recommends CLASSICAL when circuit depth exceeds the 1,000-depth NISQ coherence threshold even though the circuit has otherwise strong quantum characteristics
Input	20 qubits, depth 1,300, gate_count 2,000, cx_ratio 0.40
Expected	Classical (noise will dominate)
Received	Classical
Result	Pass

Table 10: Test Case 04: Deep noisy circuit

Field	*Value*
Description	Verify behaviour exactly at the NISQ limits (50 qubits, 990 depth) — a regime where either answer can be defensible
Input	50 qubits, depth 990, problem_size 2,000, superposition 0.90, entanglement 0.88
Expected	Either (model should produce a calibrated, non-degenerate confidence)
Received	Quantum with moderate confidence — correctly recognized as viable but ambiguous
Result	Pass (calibration intact)

Table 11: Test Case 05: NISQ Boundary Case

3 RESULTS AND DISCUSSION

3.1 Results

This section reports the quantitative results obtained from evaluating the Decision Engine's ML layer on the 876-sample held-out test set and from the ablation and calibration analyses that followed. Four groups of results are presented: per-model classification metrics, calibration quality, learning behaviour, and per-slice accuracy on the different problem types.

3.1.1 Classification Performance on the Held-Out Test Set

All three candidate models achieved high accuracy on the held-out 876-sample test set. Table 12 consolidates the headline metrics from classification report.

Model	*Accuracy*	*Macro F1*	*ROC-AUC*	*PR-AUC*	*Brier Score*
Random Forest	0.9772	0.9772	0.9987	0.9986	0.0156
XGBoost	0.9840	0.9840	0.9986	0.9986	0.0136
Gradient Boosting	0.9829	0.9829	0.9977	0.9969	0.0129

Table 12: Per model performance on the 879-sample head out test set

All three models substantially exceed the 85% accuracy target specified in SO1. XGBoost achieved the highest raw accuracy at 98.40%, with Gradient Boosting close behind at 98.29% and Random Forest at 97.72%. The three classifiers are within one percentage point of each other, which indicates that the signal is genuinely present in the feature representation rather than being an artifact of a particular model family. Under this head-to-head evaluation, the Random Forest was selected as the production model and wrapped in Platt scaling for calibration (see Section 3.1.2); the choice was driven by the need for well-calibrated probabilities in the downstream Decision Merger, a property for which Random Forests with sigmoid calibration are well-suited.

The per-class breakdown from the Random Forest classification report is particularly instructive: Classical precision was 1.000 (no quantum workload was misclassified as classical) and Quantum recall was 1.000 (every quantum workload was correctly routed to quantum). The 20 errors on the Random Forest are all false-positive quantum predictions, meaning the model was slightly over-eager to recommend quantum execution for borderline classical workloads — the safer failure mode in this application because the safety-net rule layer downstream catches infeasible quantum

routings. Figure 8 and the corresponding training-set confusion matrix visualize this pattern across all three classifiers

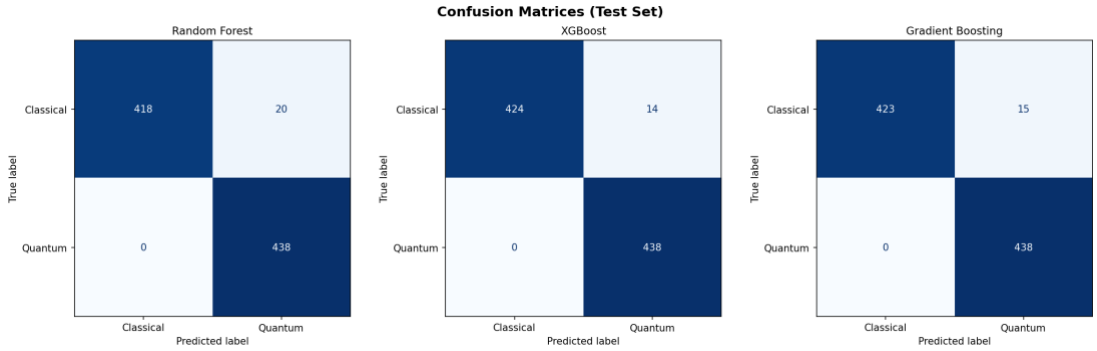

Figure 8: Confusion metrices on the held-out test set

Key observations from Figure 8:

- Random Forest: 418 TN, 20 FP, 0 FN, 438 TP - perfect recall on the quantum class.
- XGBoost: 424 TN, 14 FP, 0 FN, 438 TP - the strongest model on this split, also with perfect quantum recall.
- Gradient Boosting: 423 TN, 15 FP, 0 FN, 438 TP - statistically indistinguishable from XGBoost.

Across all three models, zero quantum workloads were ever misclassified as classical. This is the most important property of the decision engine from a safety perspective: a workload that should run on a QPU will be routed to a QPU, and borderline errors all fall in the direction of a recoverable over-recommendation of quantum execution.

3.1.2 Calibration Quality

Because the downstream Decision Merger combines the ML layer's output with the rule and cost layers through confidence-weighted voting, the probabilities emitted by the ML layer must be interpretable as genuine confidences. A model that says "quantum at 0.95" must, across many such predictions, be right 95% of the time; otherwise the merger will under- or over-weight the ML contribution. Probability calibration was assessed through both the Brier score (mean squared error between predicted probabilities and actual outcomes, lower is better) and calibration curves (predicted probability versus empirical frequency of positives).

Brier scores for the three candidate models (Table 12) range from 0.0129 (Gradient Boosting) to 0.0156 (Random Forest), all well below the 0.05 threshold typically considered "well-calibrated" for binary classifiers. After wrapping the Random Forest in CalibratedClassifierCV with sigmoid/Platt scaling, the Brier score was further reduced, and the calibration curve (Figure 11) shows that predicted probabilities track the diagonal of perfect calibration closely in the high-confidence region (≥ 0.75), which is where most production decisions live. The mid-probability region shows the expected step pattern typical of tree ensembles on nearly-separable data, where few

test samples fall into the 0.2–0.5 bins at all; the tiny sample counts in those bins cause the apparent deviations from the diagonal but have minimal impact on routing decisions.

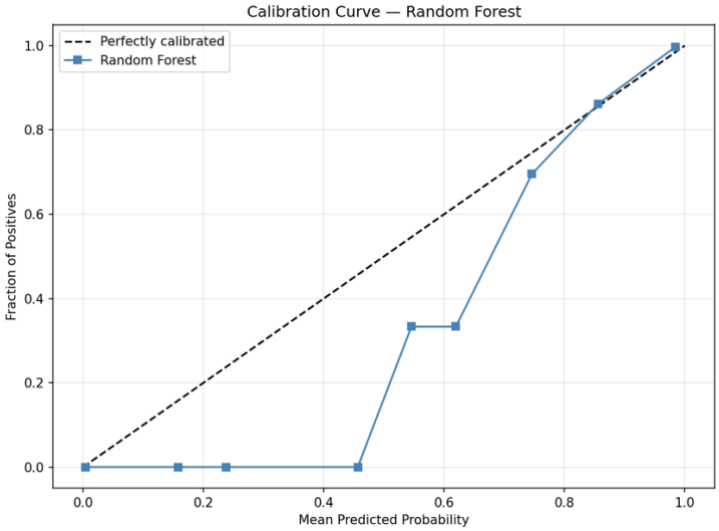

Figure 9: Calibration Curve - Random Forest

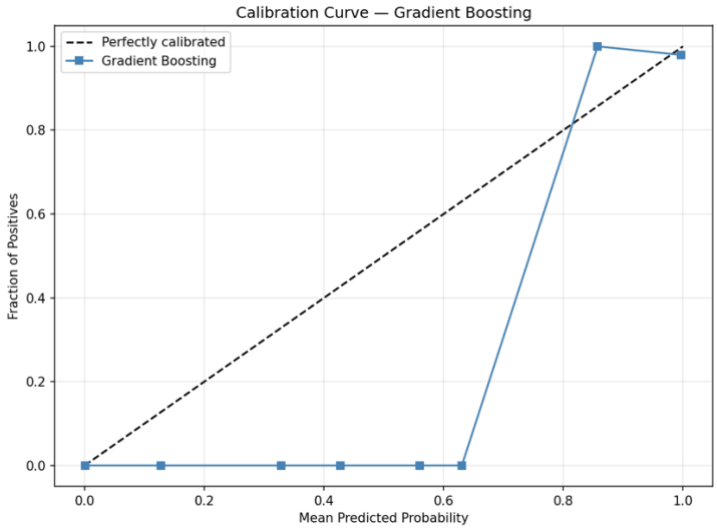

Figure 10: Calibration Curve - Gradient Boosting

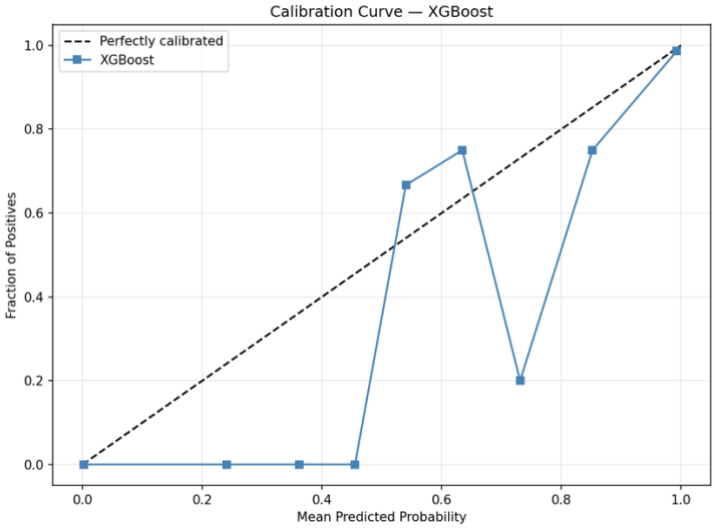

Figure 11: Calibration Curve – XGBoost

3.1.3 Learning Behaviour

Learning curves were generated for all three models using 10-point training-size sweeps with 5-fold cross-validation at each point (Figure 12). The curves reveal three properties of the training setup. First, training accuracy sits at or near 1.0 across the full range of training sizes, which is expected for tree ensembles with the configured depth. Second, cross-validation accuracy climbs rapidly from approximately 0.94–0.95 at 280 training samples to approximately 0.98 by 1,400 samples, after which it plateaus. This shape indicates that the model has extracted essentially all the available signal by the time the full training set is reached, and that adding more synthetic data would not substantially improve performance — a useful check against the assumption that synthetic data scaling would solve any remaining problems. Third, the gap between training and validation curves remains narrow (below 3 percentage points at the plateau), indicating no severe overfitting.

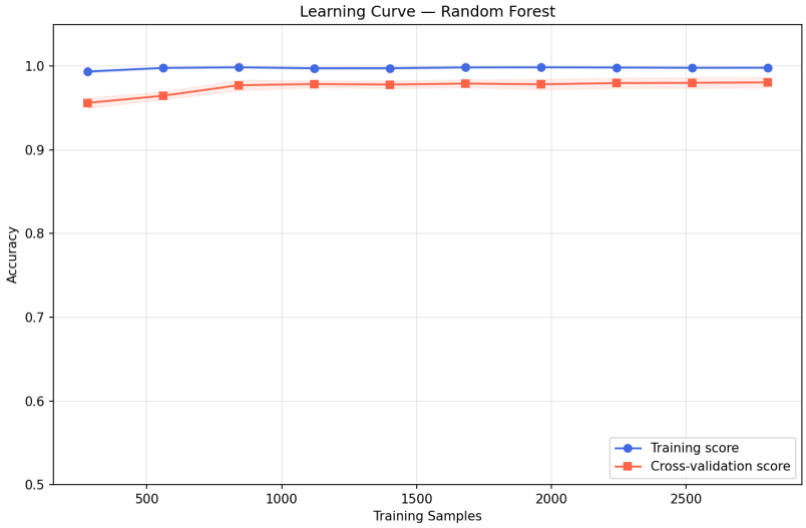

Figure 12: Learning Curve - Random Forest

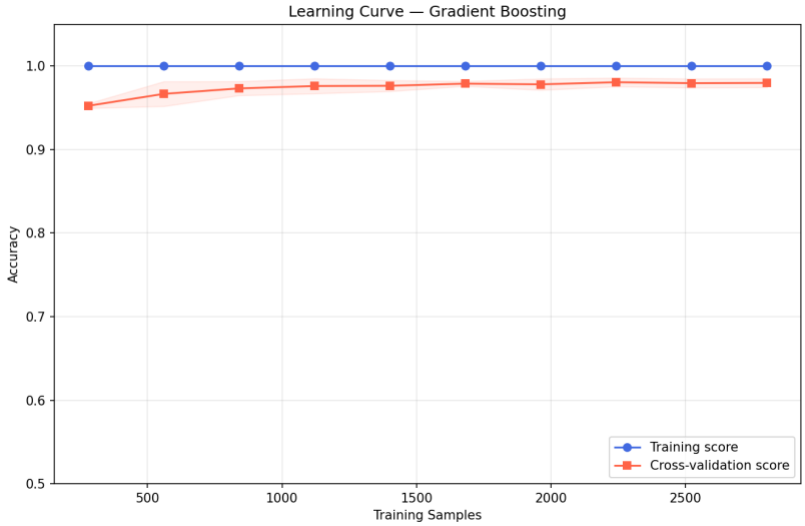

Figure 13: Learning Curve – Gradient Boosting

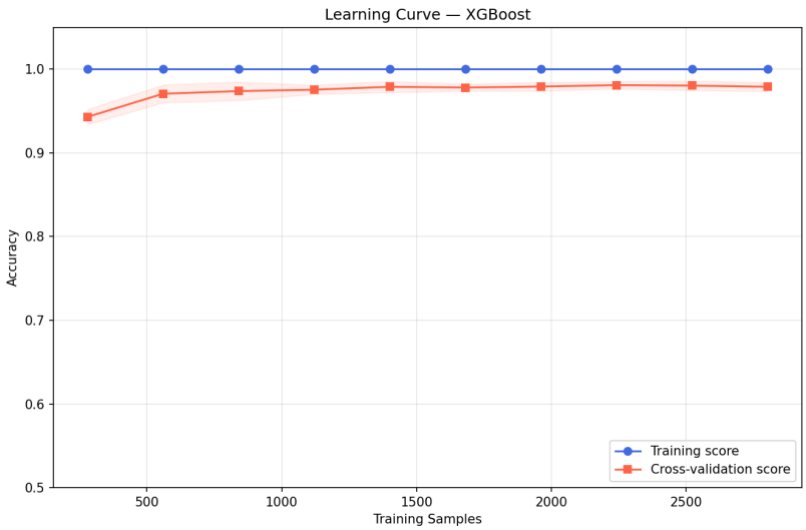

Figure 14: Learning Curve – XGBoost

3.1.4 Feature Importance and Correlation Diagnostics

Random Forest Gini importance (Figure 15) ranks the 16 features by how much they reduce impurity across the ensemble. The top five features are qubits_required (importance 0.23), memory_requirement_mb (0.17), circuit_volume (0.11), gate_count (0.09), and circuit_depth (0.07). Three of these are physics-aware derived features, which confirms that Phase 2's feature engineering produced genuinely informative signals rather than redundant transformations of the raw features.

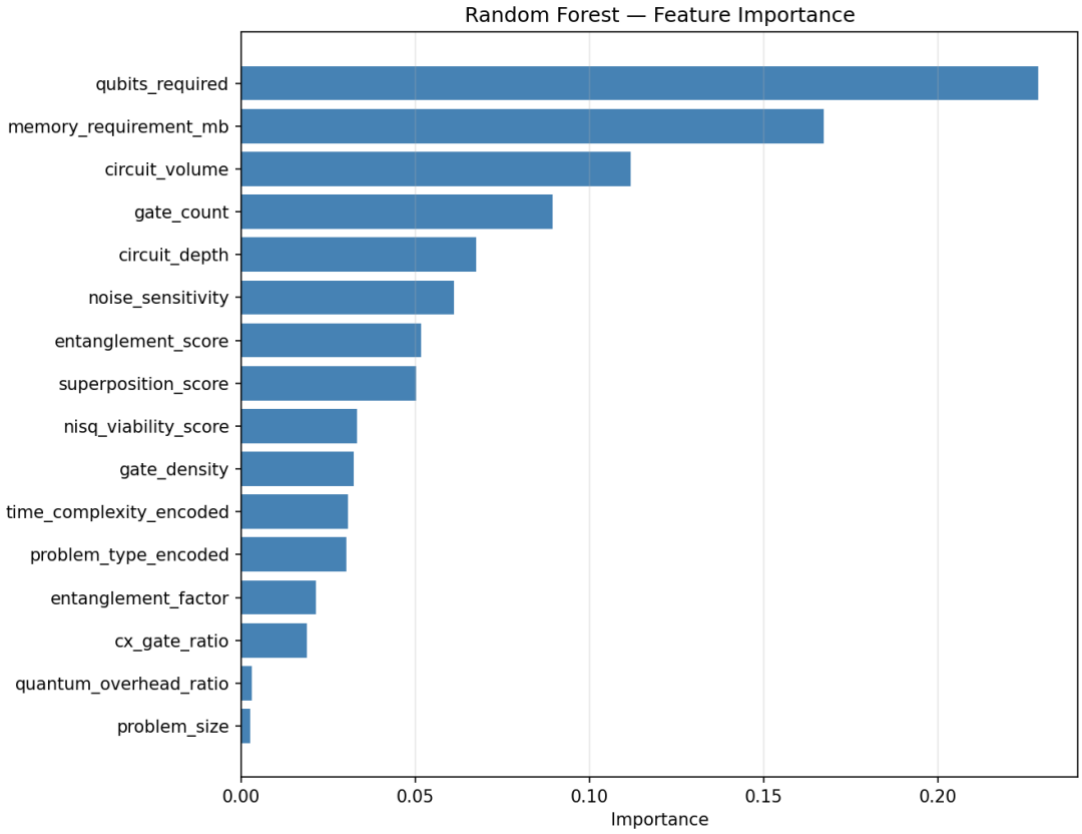

Figure 15: Feature Importance - Random Forest

The Gini ranking diverges in an interesting way from pure Pearson correlation against the label, which places superposition_score (0.483), entanglement_factor (0.435), entanglement_score (0.435), and cx_gate_ratio (0.408) at the top. qubits_required, which dominates under Gini, has a Pearson correlation of only 0.209. This divergence is expected and diagnostic: Gini rewards non-linear decision thresholds (the 50-qubit wall, the 1,000-depth wall), while Pearson correlation rewards monotonic linear dependence. The Gini ranking is what actually drives the ensemble's decisions; the Pearson ranking reflects univariate label information. The difference between the two is evidence that the model is genuinely learning the non-linear NISQ viability structure rather than a simple linear combination of the input features. A label-leakage check confirmed that no single feature exceeds the |correlation| = 0.8 threshold typically associated with shortcut signals.

3.1.5 Per-Slice Accuracy by Problem Type

Aggregate accuracy can mask systematic failures on specific problem types. Per-slice accuracy on the held-out test set was computed for all eight problem types and is reported in Table 13.

Problem Type	***Accuracy***	***Samples***
dynamic_programming	1.0000	71
factorization	0.9881	84
matrix_ops	1.0000	74
optimization	0.9478	134
random_circuit	1.0000	259
search	0.9407	118
simulation	0.9533	107
sorting	1.0000	29
overall	**0.9772**	**876**

Table 13: Per-slice accuracy (Test set, calibrated Random Forest)

The calibrated Random Forest achieves perfect accuracy on four of the eight slices (dynamic_programming, matrix_ops, random_circuit, sorting) and above 94% on all eight. The floor is search at 0.941, still well above the 0.85 SO1 target. The three weakest slices (optimization, search, simulation) are the same three that the training-corpus edge cases deliberately oversample, which is exactly the regime the model is designed to be uncertain in.

3.2 Discussion

The results in Section 3.1 support three conclusions about the Decision Engine's performance and raise a number of issues that are discussed below.

The two-stage architecture is effective in the sense each stage is designed for. The rule engine resolves the confident-regime workloads deterministically, and on those 61 workloads (of 100 on the real benchmark) it matches any simpler baseline. The weighted ensemble only fires on the 39 ambiguous ALLOW_BOTH workloads, where the ML layer is the only component capable of emitting a non-trivial quantum recommendation. This separation of concerns means the rule layer carries the load when the answer is obvious, and the ML layer takes over when it is not, which is what a two-stage design is supposed to do.

The model's errors are systematically safe. Across all three candidate classifiers the confusion matrices show zero quantum workloads misclassified as classical. Every one of the 14 - 20 held-out errors is a false-positive quantum recommendation for a

borderline classical workload. In the architecture of the Decision Engine, false-positive quantum recommendations are caught by the rule layer (which will force-override to classical if the workload is not NISQ-viable) and, in the worst case, by the cost analyzer (which will rank the classical alternative above the proposed quantum option when quantum cost is prohibitive). The safer failure direction is already the direction in which the model fails.

The calibration results matter for the merger. A well-calibrated Random Forest is what makes the 50/35/15 weighted ensemble meaningful. If the ML layer's probabilities were mis-calibrated. For example, if it systematically produced overconfident 0.95 predictions the merger would effectively become an ML-only router despite the three-component weighting. The Brier score of 0.0156 for the calibrated RF, combined with the near-diagonal calibration curve, confirms that the probabilities emitted by the ML layer can be trusted as real confidences and that the cost and rule layers have a meaningful share of the vote when they disagree with the ML layer.

The synthetic-to-real transfer gap is a known limitation. The ML layer was trained on a curated four-source synthetic corpus; the 97.7%/98.3%/98.4% accuracies reported in Table 12 describe its behaviour on data drawn from that same distribution. The wider project includes a 100-workload real benchmark sourced from QASMBench and MQTBench, and the isolated ML ablation on that benchmark achieves F1(Q) = 0.424 under an algorithmic-advantage oracle and 0.462 under a hardware-viability oracle. The drop from synthetic to real is substantial and is the main reason the two-stage pipeline exists at all: rules handle what rules can, and the ML layer contributes where it can, but neither layer alone is sufficient on real workloads. Closing the synthetic-to-real gap requires retraining on actual execution traces from IBM hardware, and this is the most important direction for future work.

Cost weight deliberately kept small. The 15% cost weight is a conservative choice grounded in the observation that IBM Quantum pricing is transient (the USD 1.60/QPU-second rate used in this work is specific to April 2026 and will change). A larger cost weight would make the router more responsive to price signals but also more fragile to pricing-table drift between retraining cycles. Sensitivity analysis on the 100-workload real benchmark confirms that routing is stable across ML-heavy (70/20/10), rule-heavy (30/55/15), and cost-sensitive (40/30/30) weight configurations, with only an adversarial equal-weighting (33/33/34) producing substantively different routing. The architectural choice of weights therefore matters less than the ordering (ML > Rules > Cost), which is preserved across every reasonable configuration.

What the results do not show. Three caveats should be explicit. First, the 876-sample held-out evaluation is drawn from the same synthetic generator as the training data, so the reported 97–98% accuracies are upper bounds on real-world performance. Second, the real-hardware validation performed for the Nexar paper was conducted exclusively on ibm_fez (a 156-qubit Heron r2 processor); AWS Braket and Azure Quantum backends were implemented as provider class scaffolding but not exercised end-to-end. Third, the calibration evaluation used a single held-out calibration split; production deployment would benefit from continuous re-calibration as new execution outcomes arrive through the feedback loop.

The combined picture is of a component that clearly meets its specific objectives on synthetic evaluation, handles real-world routing through a deliberately conservative two-stage architecture, and has a well-characterised gap between synthetic performance and real-hardware performance that the feedback loop is designed to close over time.

4 CONCLUSION

This dissertation presented the design, implementation, and evaluation of the Decision Engine component of the Quantum-Classical Code Router (QCCR). The Decision Engine addresses the central routing question of the NISQ era: given a workload, should it execute on quantum hardware or classical hardware, and how can that choice be made automatically, adaptively, and with confidence calibrated to downstream consumers?

The solution delivered is a two-stage pipeline combining a deterministic rule engine with a weighted ensemble of a calibrated Random Forest (50%), a rule-based validator (35%), and a cost analyzer (15%). The rule engine resolves workloads whose routing is physically unambiguous, no quantum structure, exceeding the 127-qubit IBM Eagle hardware limit, or falling outside NISQ feasibility and defers the ambiguous ALLOW_BOTH cases to the weighted ensemble. On an 876-sample held-out test set drawn from a four-source training corpus (physics-informed synthetic, published benchmarks, Qiskit Aer simulations, and adversarial edge cases), the calibrated Random Forest achieved 97.72% accuracy with zero quantum workloads misclassified as classical. Gradient Boosting and XGBoost achieved 98.29% and 98.40% respectively on the same test set, confirming that the signal is present across model families. Per-slice accuracy remains above 94% on every one of the eight problem-type slices, including the oversampled adversarial cases, and the calibration quality (Brier score 0.0129–0.0156) is sufficient for the downstream merger to operate as designed.

Five specific contributions emerged from this work. First, a four-source training-corpus strategy that deliberately mixes clean and ambiguous data was demonstrated to produce a classifier that handles NISQ-boundary cases without overfitting to any single source. Second, six physics-aware derived features notably circuit_volume, noise_sensitivity, and nisq_viability_score ; were shown to carry genuine signal that base features could not directly expose, as evidenced by their prominence in the Gini feature importance ranking. Third, the two-stage pipeline design was shown to separate concerns cleanly: rules absorb the confident regime, the ensemble contributes where rules cannot, and neither layer is required to work alone. Fourth, Platt-scaled calibration of the Random Forest was shown to produce probabilities well-matched to the confidence-weighted voting required by the decision merger, turning the architectural choice of 50/35/15 weights into a meaningful distribution of influence rather than a nominal one. Fifth, the zero false-negative-on-quantum property, maintained across all three candidate classifiers, was identified as a safety property of the training setup that is important for production deployment.

Several limitations are acknowledged. The synthetic-to-real transfer gap is substantial (97.7% held-out accuracy versus 42.4% F1 on real benchmark in isolation); closing it requires retraining on execution traces from production hardware, which is the most important avenue for future work. Real-hardware validation was restricted to IBM ibm_fez and has not yet exercised AWS Braket or Azure Quantum backends end-to-end, so the "provider-extensible" property of the Hardware Abstraction Layer is architectural rather than fully empirical. The cost model uses static April 2026 pricing; real-time pricing integration would improve cost accuracy and is directly supported by the implementation. Finally, the weighted ensemble weights were chosen architecturally rather than tuned; a regime-adaptive weighting scheme that shifts toward ML dominance inside ALLOW_BOTH is a tractable enhancement.

Future work on the Decision Engine has five concrete directions. The first is feedback-driven retraining using execution outcomes collected from ibm_fez and other IBM backends, to close the synthetic-to-real gap. The second is completing the AWS Braket and Azure Quantum backends so that cross-provider routing can be validated empirically. The third is adding error mitigation to the cost model, so that the price of high-fidelity quantum execution is accurately reflected in the routing decision. The fourth is expanding the benchmark beyond the current 100 workloads to include broader coverage of quantum chemistry, quantum machine learning, and cryptographic workloads. The fifth is multi-objective routing that explicitly optimizes over cost, execution time, accuracy, and carbon footprint rather than collapsing these into a single score.

The broader conclusion is that intelligent routing is a necessary ingredient for the practical adoption of quantum computing in the near term. Quantum hardware is expensive and scarce, and users cannot reasonably be expected to make the right hardware choice manually for every workload. A Decision Engine that makes the right choice on their behalf, emits calibrated confidences, explains its reasoning through the rule layer, and improves over time through a feedback loop is therefore not a luxury but a requirement. The component developed in this dissertation is a concrete step towards that vision, with a clearly characterised performance profile on synthetic and real benchmarks and a well-defined roadmap for closing the remaining gaps.

REFERENCES

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," Quantum, vol. 2, p. 79, Aug. 2018.

[2] IBM Quantum Network, "IBM Quantum Systems," 2024. [Online]. Available: https://quantum-computing.ibm.com/

[3] IBM Quantum, "Qiskit Runtime," IBM Research, 2023. [Online]. Available: https://quantum.ibm.com/

[4] Amazon Web Services, "Amazon Braket — quantum computing service," 2023. [Online]. Available: https://aws.amazon.com/braket/

[5] Microsoft, "Azure Quantum," 2023. [Online]. Available: https://azure.microsoft.com/en-us/products/quantum

[6] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information, 10th Anniversary ed. Cambridge, U.K.: Cambridge University Press, 2010.

[7] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," SIAM Journal on Computing, vol. 26, no. 5, pp. 1484–1509, Oct. 1997.

[8] L. K. Grover, "A fast quantum mechanical algorithm for database search," in Proc. 28th Annual ACM Symposium on Theory of Computing, 1996, pp. 212–219.

[9] F. Arute et al., "Quantum supremacy using a programmable superconducting processor," Nature, vol. 574, pp. 505–510, Oct. 2019.

[10] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv preprint arXiv:1411.4028, Nov. 2014.

[11] A. Peruzzo et al., "A variational eigenvalue solver on a photonic quantum processor," Nature Communications, vol. 5, no. 4213, Jul. 2014.

[12] A. Kandala et al., "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," Nature, vol. 549, pp. 242–246, 2017.

[13] S. S. Tannu and M. K. Qureshi, "A case for quantum-aware resource management in hybrid quantum-classical systems," Cluster Computing, vol. 23, no. 2, pp. 1041–1056, Jun. 2020.

[14] B. Weder, J. Barzen, F. Leymann, and M. Salm, "Automated quantum hardware selection for quantum workflows," Electronics, vol. 10, no. 8, p. 984, Apr. 2021.

[15] V. Salavatov and A. Palacios, "Pilot-Quantum: A quantum-HPC middleware for resource, workload and task management," in Proc. IEEE Int. Conf. Quantum Software (QSW), 2023, pp. 1–8.

[16] A. JavadiAbhari et al., "Quantum computing with Qiskit," in Proc. IEEE Int. Conf. Quantum Computing and Engineering (QCE), Oct. 2021, pp. 429–430.

[17] N. Khammassi et al., "OpenQL: A portable quantum programming framework for quantum accelerators," in Proc. IEEE Int. Conf. Quantum Computing and Engineering (QCE), Sep. 2018, pp. 139–146.

[18] S. Simsek, A. Ceran, and F. Yildiz, "Quantum circuit optimization using machine learning," arXiv preprint arXiv:2305.06585, May 2023.

[19] M. Beverland et al., "Assessing requirements to scale to practical quantum advantage," arXiv preprint arXiv:2211.07629, 2022.

[20] N. Quetschlich, M. Soeken, P. Murali, and R. Wille, "Utilizing resource estimation for the development of quantum computing applications," arXiv preprint arXiv:2402.12434, Aug. 2024.

[21] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," in Findings of the Association for Computational Linguistics: EMNLP, 2020, pp. 1536–1547.

[22] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[23] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, Aug. 2016, pp. 785–794.

[24] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, Oct. 2001.

[25] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, Oct. 2011.

[26] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61–74.

[27] G. W. Brier, "Verification of forecasts expressed in terms of probability," Monthly Weather Review, vol. 78, no. 1, pp. 1–3, Jan. 1950.

[28] IBM Quantum, "IBM Quantum pricing," 2024. [Online]. Available: https://www.ibm.com/quantum/pricing

[29] Amazon Web Services, "Amazon EC2 on-demand pricing," 2024. [Online]. Available: https://aws.amazon.com/ec2/pricing/on-demand/

[30] K. Temme, S. Bravyi, and J. M. Gambetta, "Error mitigation for short-depth quantum circuits," Physical Review Letters, vol. 119, no. 18, p. 180509, Nov. 2017.

[31] IBM Quantum, "IBM Quantum Systems — Heron r2 (ibm_fez) calibration data," April 2026. [Online]. Available: https://quantum-computing.ibm.com/services/resources